LAB 7: Fuzzing for Input Validation Bugs

Lab Report & Evidence Document | Topic: Property-Based Fuzz Testing with Hypothesis

Name : Shantheri Shenoy

SRN : PES1UG24CS428

Section : H

1. Objective & Learning Outcomes

The objective of this lab is to explore fuzz testing as an automated security and quality assurance technique. By leveraging the Hypothesis property-based testing library in Python, we uncover hidden edge cases, unhandled exceptions, and input-validation vulnerabilities in application code, and harden the software through defensive programming.

- Key Learning Outcomes Achieved:
    - Understood the core methodology and benefits of property-based fuzzing over traditional static unit testing.
    - Applied property testing decorators (@given) and strategies (st.text(), st.none()) using Hypothesis.
    - Detected, minimized, and analyzed crashing edge-case inputs (NoneType, malformed strings, out-of-bound indices).
    - Hardened application routines with comprehensive defensive validation and fail-safe exception boundaries.

2. Step 1: Installing Dependencies

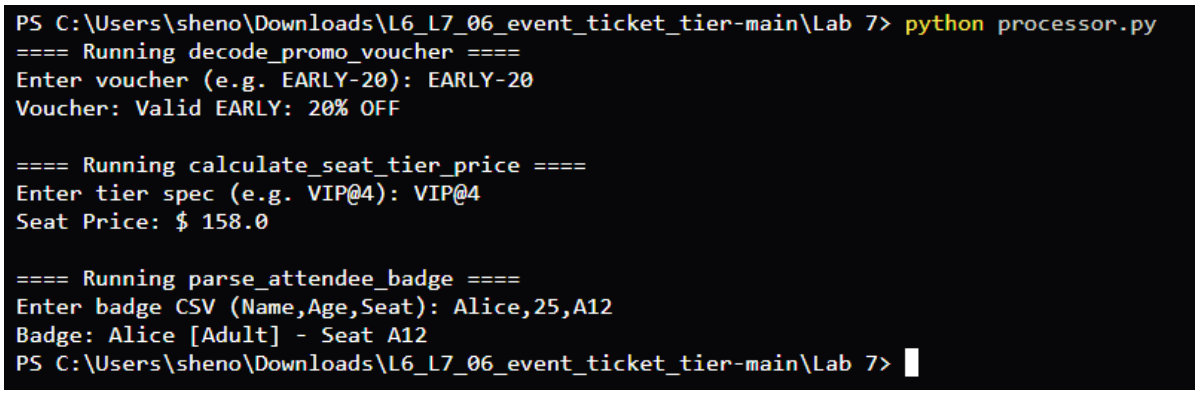
The required fuzz testing and test runner packages were installed in the working environment:

```
pip install hypothesis
pip install pytest
```

3. Step 2: Running the Buggy Application (processor.py)

The initial processor script was executed under Lab 7 using nominal sample inputs to establish the baseline functionality. As expected, under normal, well-formed inputs the script operates without immediate error:

```
PS C:\Users\sheno\Downloads\L6_L7_06_event_ticket_tier-main\Lab 7> python processor.py
```



> **Observation:** While the script succeeds on expected inputs like 'EARLY-20', 'VIP@4', and 'Alice,25,A12', it assumes inputs are always correctly delimited and strictly typed, making it fragile against malformed data, empty strings, and null inputs.

4. Step 3 & 4: Property-Based Fuzz Testing & Bug Detection

Implementation of test_processor.py using Hypothesis

To systematically probe for crashes across wide input domains, property tests were written in test_processor.py using the @given decorator combined with st.text() | st.none(). Each test asserts failure if an unhandled exception is raised:

```
from hypothesis import given, strategies as st
from processor import decode_promo_voucher, calculate_seat_tier_price, parse_attendee_badge

@given(st.text() | st.none())
def test_decode_promo_voucher_no_crash(s):
    try:
        decode_promo_voucher(s)
    except Exception as e:
        assert False, f'decode_promo_voucher crashed with: {e}'

@given(st.text() | st.none())
def test_calculate_seat_tier_price_safe(s):
    try:
        calculate_seat_tier_price(s)
    except Exception as e:
        assert False, f'calculate_seat_tier_price crashed with: {e}'

@given(st.text() | st.none())
def test_parse_attendee_badge_safe(s):
    try:
        parse_attendee_badge(s)
    except Exception as e:
        assert False, f'parse_attendee_badge crashed with: {e}'
```

Fuzz Testing Execution & Uncovered Failures

The test suite was executed against the initial processor code:

```
PS C:\Users\sheno\Downloads\L6_L7_06_event_ticket_tier-main\Lab 7> pytest test_processor.py
```

```
=========================================== short test summary info ============================================
FAILED test_processor.py::test_decode_promo_voucher_no_crash - ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
FAILED test_processor.py::test_calculate_seat_tier_price_safe - ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
FAILED test_processor.py::test_parse_attendee_badge_safe - ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
============================================== 3 failed in 1.55s ===============================================
PS C:\Users\sheno\Downloads\L6_L7_06_event_ticket_tier-main\Lab 7>
```

Root Cause Analysis of Discovered Bugs

Hypothesis rapidly generated minimal falsifying examples that exposed critical input-validation vulnerabilities:

Function Under Test	Failing Input (Shrunk)	Root Cause & Unhandled Exception
decode_promo_voucher	s = None	AttributeError on .strip(); ValueError (not enough values to unpack) when '-' delimiter is missing; ValueError on non-int

	s = ''	discount.
calculate_seat_tier_price	s = None s = 'VIP'	AttributeError on .split('@'); IndexError when '@' delimiter is absent; KeyError when tier is unlisted; ValueError on invalid row.
parse_attendee_badge	s = None s = ''	AttributeError on .split(','); IndexError when 3 comma-separated tokens are not provided; ValueError when age is non-numeric.

5. Step 5: Fixing & Hardening processor.py

To achieve robust defensive programming, processor.py was rewritten. Each function implements explicit type checking, delimiter and token length validation, safe conversion parsing, logical boundary checks, and a top-level try/except boundary to guarantee that no malformed input can cause an unhandled crash.

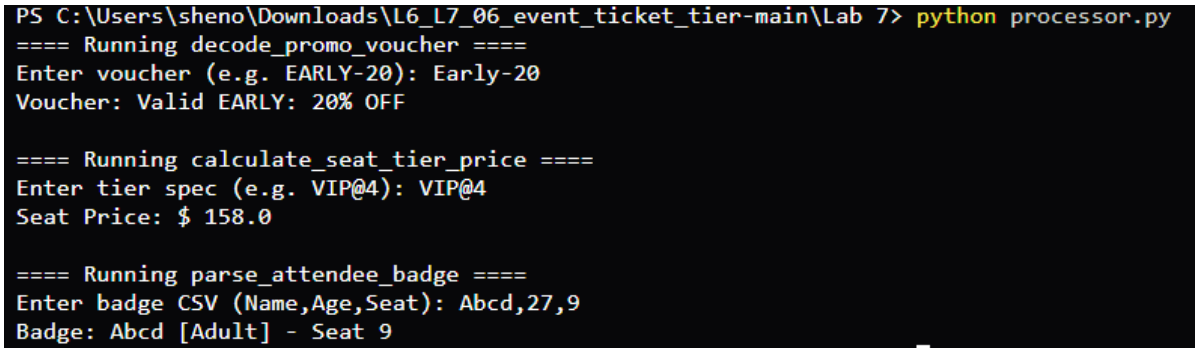
```
def decode_promo_voucher(voucher_code):
    try:
        if not isinstance(voucher_code, str):
            return 'Invalid voucher'
        code = voucher_code.strip()
        if '-' not in code:
            return 'Invalid voucher format'
        parts = code.split('-')
        if len(parts) != 2:
            return 'Invalid voucher format'
        prefix, pct_str = parts[0].strip(), parts[1].strip()
        if not prefix:
            return 'Invalid voucher prefix'
        percentage = int(pct_str)
        if percentage < 0 or percentage > 100:
            return 'Invalid percentage'
        return f'Valid {prefix.upper()}: {percentage}% OFF'
    except Exception:
        return 'Invalid voucher'

def calculate_seat_tier_price(tier_spec):
    try:
        if not isinstance(tier_spec, str):
            return 0.0
        parts = tier_spec.split('@')
        if len(parts) != 2:
            return 0.0
        tier = parts[0].strip().upper()
        row = int(parts[1].strip())
        if row < 0:
            return 0.0
        tier_rates = {'VIP': 150.0, 'PREMIUM': 90.0, 'GENERAL': 45.0}
        if tier not in tier_rates:
            return 0.0
        base = tier_rates[tier]
        return base + (row * 2.0)
    except Exception:
        return 0.0
```

```
def parse_attendee_badge(badge_csv):
    try:
        if not isinstance(badge_csv, str):
            return 'Invalid badge'
        tokens = badge_csv.split(',')
        if len(tokens) != 3:
            return 'Invalid badge format'
        name = tokens[0].strip()
        if not name:
            return 'Invalid name'
        age = int(tokens[1].strip())
        if age < 0 or age > 150:
            return 'Invalid age'
        seat = tokens[2].strip()
        if not seat:
            return 'Invalid seat'
        category = 'Child' if age < 18 else 'Adult'
        return f'{name} [{category}] - Seat {seat}'
    except Exception:
        return 'Invalid badge'
```

The fixed script was run to ensure expected nominal operation remains fully intact:

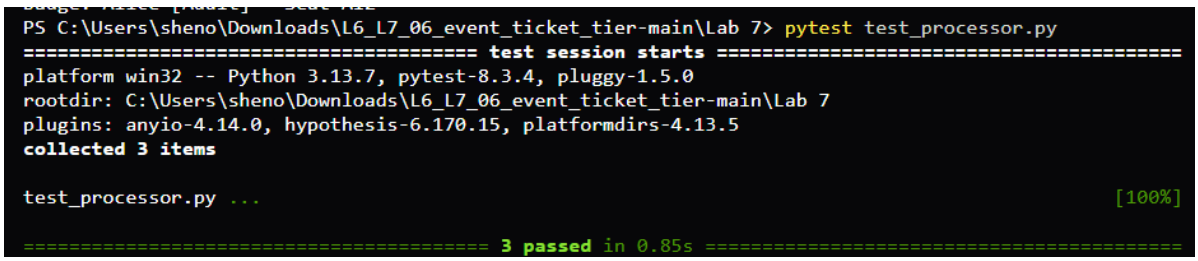
```
PS C:\Users\sheno\Downloads\L6_L7_06_event_ticket_tier-main\Lab 7> python processor.py
```



6. Step 6: Test Re-Execution & Verification

With the defensive validations in place, pytest was re-executed. Hypothesis subjected the functions to hundreds of randomized fuzz inputs (including nulls, empty strings, Unicode, extreme integers, and malformed delimiters).

```
PS C:\Users\sheno\Downloads\L6_L7_06_event_ticket_tier-main\Lab 7> pytest test_processor.py
```



7. Conclusion

Fuzz testing with Hypothesis proved highly effective in uncovering edge-case crashes that standard happy-path unit tests consistently miss. By moving from example-based testing to property-based fuzzing, we verified that the functions hold their invariants (no unhandled exceptions) across the entire input space. Implementing defensive programming practices safeguarded the application against denial-of-service, unhandled crashes, and malformed payloads.